Bitswap 데이터 교환

개요

Bitswap은 IPFS의 핵심 데이터 교환 프로토콜로, 피어 간에 블록 데이터를 효율적으로 주고받는 역할을 합니다. 클라이언트가 요청한 블록을 네트워크에서 획득하고, 자신이 보유한 블록을 다른 피어에게 제공하는 두 가지 주요 기능을 수행합니다.

Bitswap은 요청-응답 방식이 아닌 메시지 기반 프로토콜로 동작하며, 모든 메시지는 원하는 블록 목록(wantlist) 또는 실제 블록 데이터를 포함합니다.

주요 개념

Wantlist (원하는 블록 목록)

- 노드가 원하는 블록들의 CID 목록
- 다른 피어에게 어떤 블록이 필요한지 알려주는 역할
- 블록을 받으면 해당 블록에 대한 Cancel 메시지를 전송

세션 (Session)

- 관련된 블록 요청들을 그룹화하여 관리
- 동일한 피어 그룹에서 연속적인 요청을 효율적으로 처리
- MerkleDAG 구조의 파일을 가져올 때 성능 최적화

블록 발견 (Block Discovery)

- 연결된 피어들에게 `want-have` 메시지 브로드캐스트

- DHT를 통한 블록 제공자 검색
- 피어 선택 알고리즘을 통한 최적 피어 결정

기본 사용법

Bitswap 초기화

```
package main

import (
"context"
"github.com/ipfs/boxo/bitswap"
bsnet "github.com/ipfs/boxo/bitswap/network"
"github.com/ipfs/boxo/blockstore"
"github.com/ipfs/go-datastore"
dsync "github.com/ipfs/go-datastore/sync"
"github.com/libp2p/go-libp2p/core/host"
"github.com/libp2p/go-libp2p/core/routing"
)

func initializeBitswap(ctx context.Context, h host.Host, router routing.Content
// 데이터스토어와 블록스토어 생성
ds := dsync.MutexWrap(datastore.NewMapDatastore())
bstore := blockstore.NewBlockstore(ds)
```

```
// 네트워크 인터페이스 생성
network := bsnet.NewFromIpfsHost(h, router)
// Bitswap 인스턴스 생성
exchange := bitswap.New(ctx, network, router, bstore)
return exchange
}
```

단일 블록 동기 요청

```
func getBlockSync(ctx context.Context, exchange *bitswap.Bitswap, c cid.Cid) (b
// 특정 CID의 블록을 동기적으로 요청
block, err := exchange.GetBlock(ctx, c)
if err != nil {
return nil, fmt.Errorf("블록 가져오기 실패: %w", err)
}
log.Printf("블록 수신 완료: %s, 크기: %d bytes", c, len(block.RawData()))
return block, nil
}
```

여러 블록 비동기 요청

```
func getBlocksAsync(ctx context.Context, exchange *bitswap.Bitswap, cids []cid.
```

```
// 여러 블록을 비동기적으로 요청

blockChannel, err := exchange.GetBlocks(ctx, cids)

if err != nil {

return fmt.Errorf("블록들 요청 실패: %w", err)

}

// 채널에서 블록들을 수신

receivedCount := 0

for block := range blockChannel {

log.Printf("블록 수신: %s", block.Cid())

receivedCount++

// 모든 블록을 받았으면 종료

if receivedCount == len(cids) {

break

}

}

return nil

}
```

고급 활용

세션을 이용한 관련 블록 요청

```
func fetchRelatedBlocks(ctx context.Context, exchange *bitswap.Bitswap, rootCid
```

```
// 새로운 세션 생성 - 관련된 블록들을 효율적으로 가져오기 위함

session := exchange.NewSession(ctx)

// 루트 블록 먼저 가져오기

rootBlock, err := session.GetBlock(ctx, rootCid)

if err != nil {

return fmt.Errorf("루트 블록 가져오기 실패: %w", err)

}

// 루트 블록에서 링크된 CID들 추출 (예시)

linkedCids := extractLinkedCids(rootBlock)

// 같은 세션에서 관련 블록들 요청

blockChannel, err := session.GetBlocks(ctx, linkedCids)

if err != nil {

return fmt.Errorf("관련 블록들 요청 실패: %w", err)

}

// 블록들 처리

for block := range blockChannel {

log.Printf("관련 블록 수신: %s", block.Cid())

// 블록 처리 로직

}

return nil

}



// 블록에서 링크된 CID들을 추출하는 헬퍼 함수 (구현 예시)
```

```
func extractLinkedCids(block blocks.Block) []cid.Cid {
// 실제 구현에서는 블록 타입에 따라 적절한 파싱 로직 필요
var cids []cid.Cid
// DAG-PB, DAG-CBOR 등의 형식에 따른 링크 추출 로직
return cids
}
```

새로운 블록 알림

```
func notifyNewBlocks(ctx context.Context, exchange *bitswap.Bitswap, blocks []b
// 로컬 데이터스토어에 새로운 블록이 추가되었음을 Bitswap에 알림
err := exchange.NotifyNewBlocks(ctx, blocks...)
if err != nil {
return fmt.Errorf("새 블록 알림 실패: %w", err)
}
log.Printf("%d개의 새 블록을 Bitswap에 알림", len(blocks))
return nil
}
```

세션 관리 및 최적화

세션 생명주기 관리

```
type SessionManager struct {
exchange *bitswap.Bitswap
sessions  map[string]exchange.Fetcher
mu sync.RWMutex
}


func  NewSessionManager(exchange *bitswap.Bitswap) *SessionManager {
return &SessionManager{
exchange: exchange,
sessions: make(map[string]exchange.Fetcher),
}
}


func (sm *SessionManager) GetOrCreateSession(ctx context.Context, sessionID  str
sm.mu.Lock()
defer  sm.mu.Unlock()
if  session, exists := sm.sessions[sessionID];  exists {
return  session
}
// 새 세션 생성
session := sm.exchange.NewSession(ctx)
sm.sessions[sessionID] = session
```

```
log.Printf("새 세션 생성: %s", sessionID)

return session

}


func (sm *SessionManager) CloseSession(sessionID string) {

sm.mu.Lock()

defer sm.mu.Unlock()

delete(sm.sessions[sessionID])

log.Printf("세션 종료: %s", sessionID)

}
```

성능 모니터링

```
func monitorBitswapStats(exchange *bitswap.Bitswap) {

ticker := time.NewTicker(30 * time.Second)

defer ticker.Stop()

for range ticker.C {

stats, err := exchange.Stat()

if err != nil {

log.Printf("통계 조회 실패: %v", err)

continue

}

log.Printf("Bitswap 통계:")
```

```
log.Printf(" 대기 중인 블록: %d", len(stats.Wantlist))
log.Printf(" 연결된 피어: %d", len(stats.Peers))
log.Printf(" 수신한 블록: %d (총 %d bytes)", stats.BlocksReceived, stats.DataRecei
log.Printf(" 전송한 블록: %d (총 %d bytes)", stats.BlocksSent, stats.DataSent)
log.Printf(" 중복 블록: %d (총 %d bytes)", stats.DupBlksReceived, stats.DupDataRe
}
}
```

설정 옵션

서버 관련 옵션

```
func  createOptimizedBitswap(ctx context.Context, network network.BitSwapNetwork
router routing.ContentRouting, bstore blockstore.Blockstore) *bitswap.Bitswap {
options := []bitswap.Option{
// 엔진 워커 수 설정
bitswap.EngineBlockstoreWorkerCount(4),
bitswap.EngineTaskWorkerCount(8),
// 피어당 최대 대기 바이트 수
bitswap.MaxOutstandingBytesPerPeer(1 << 20), // 1MB
// 피어당 최대 대기열 항목 수
bitswap.MaxQueuedWantlistEntriesPerPeer(32),
// 최대 CID 크기 제한
```

```
bitswap.MaxCidSize(100),
// DONT_HAVE 메시지 전송 여부
bitswap.SetSendDontHaves(true),
// 메시지 크기 설정
bitswap.WithTargetMessageSize(16 * 1024), // 16KB
// WantHave를 WantBlock으로 교체할 최대 크기
bitswap.WithWantHaveReplaceSize(1024), // 1KB
}
return  bitswap.New(ctx, network, router, bstore, options...)
}
```

클라이언트 관련 옵션

```
func  createClientOptimizedBitswap(ctx context.Context, network network.BitSwapN
router routing.ContentRouting, bstore blockstore.Blockstore) *bitswap.Bitswap {
options := []bitswap.Option{
// 제공자 검색 지연 시간
bitswap.ProviderSearchDelay(1 * time.Second),
// 재브로드캐스트 지연 시간
bitswap.RebroadcastDelay(delay.Fixed(30 * time.Second)),
// 타임아웃 시 DONT_HAVE 시뮬레이션
bitswap.SetSimulateDontHavesOnTimeout(true),
// 중복 블록 통계 비활성화 (성능 향상)
```

```
bitswap.WithoutDuplicatedBlockStats(),
// 서버 기능 비활성화 (클라이언트 전용)
bitswap.WithServerEnabled(false),
}
return  bitswap.New(ctx, network, router, bstore, options...)
}
```

실제 예제: P2P 파일 전송

```
package main

import (
"bytes"
"context"
"fmt"
"io"
"log"
"github.com/ipfs/boxo/bitswap"
bsnet  "github.com/ipfs/boxo/bitswap/network"
"github.com/ipfs/boxo/blockservice"
"github.com/ipfs/boxo/blockstore"
"github.com/ipfs/boxo/chunker"
```

```
"github.com/ipfs/boxo/files"

"github.com/ipfs/boxo/ipld/merkledag"

unixfile  "github.com/ipfs/boxo/ipld/unixfs/file"

"github.com/ipfs/boxo/ipld/unixfs/importer/balanced"

uih  "github.com/ipfs/boxo/ipld/unixfs/importer/helpers"

"github.com/ipfs/go-cid"

"github.com/ipfs/go-datastore"

dsync  "github.com/ipfs/go-datastore/sync"

"github.com/libp2p/go-libp2p/core/host"

"github.com/multiformats/go-multicodec"

)



// 파일 서버: 파일을 UnixFS로 저장하고 Bitswap으로 제공

func  startFileServer(ctx context.Context, h host.Host, fileData []byte) (cid.Ci

// 블록스토어 설정

ds := dsync.MutexWrap(datastore.NewMapDatastore())

bs := blockstore.NewBlockstore(ds)

// UnixFS로 파일 임포트

fileReader := bytes.NewReader(fileData)

bsrv := blockservice.New(bs, nil)

dsrv := merkledag.NewDAGService(bsrv)

// UnixFS 임포트 매개변수 설정

ufsImportParams := uih.DagBuilderParams{
```

```
Maxlinks: uih.DefaultLinksPerBlock,

RawLeaves: true,

CidBuilder: cid.V1Builder{

Codec: uint64(multicodec.DagPb),

MhType: uint64(multicodec.Sha2_256),

MhLength: -1,

},

Dagserv: dsrv,

NoCopy: false,

}

// 파일을 청크로 분할하고 DAG 구조 생성

ufsBuilder, err := ufsImportParams.New(chunker.NewSizeSplitter(fileReader, chunk

if  err != nil {

return  cid.Undef, nil, err

}

nd, err := balanced.Layout(ufsBuilder)

if  err != nil {

return  cid.Undef, nil, err

}

// Bitswap 서버 시작

network := bsnet.NewFromIpfsHost(h, nil)

exchange := bitswap.New(ctx, network, nil, bs)

return  nd.Cid(), exchange, nil
```

```
}


// 파일 클라이언트: Bitswap을 통해 파일 다운로드
func downloadFile(ctx context.Context, h host.Host, fileCid cid.Cid, targetPeer
// 클라이언트용 블록스토어 (메모리만 사용)
bs := blockstore.NewBlockstore(datastore.NewNullDatastore())
// Bitswap 클라이언트 생성
network := bsnet.NewFromIpfsHost(h, nil)
exchange := bitswap.New(ctx, network, nil, bs)
defer exchange.Close()
// 타겟 피어에 연결
if err := connectToPeer(ctx, h, targetPeer); err != nil {
return nil, fmt.Errorf("피어 연결 실패: %w", err)
}
// DAG 서비스 생성
bsrv := blockservice.New(bs, exchange)
dsrv := merkledag.NewDAGService(bsrv)
// 루트 노드 가져오기
nd, err := dsrv.Get(ctx, fileCid)
if err != nil {
return nil, fmt.Errorf("루트 노드 가져오기 실패: %w", err)
}
// UnixFS 파일로 변환
```

```
unixFSNode, err := unixfile.NewUnixfsFile(ctx, dsrv, nd)

if err != nil {

return nil, fmt.Errorf("UnixFS 파일 생성 실패: %w", err)

}

// 파일 데이터 읽기

var buf bytes.Buffer

if f, ok := unixFSNode.(files.File); ok {

if _, err := io.Copy(&buf, f); err != nil {

return nil, fmt.Errorf("파일 읽기 실패: %w", err)

}

}

return buf.Bytes(), nil

}


func connectToPeer(ctx context.Context, h host.Host, targetPeer string) error

// 멀티어드레스 파싱 및 피어 연결 로직

// 실제 구현에서는 multiaddr 파싱과 peer.AddrInfo 생성 필요

return nil

}
```

모범 사례

1. 세션 활용

- 관련된 블록들(예: 파일의 모든 청크)을 요청할 때는 반드시 세션 사용
- 세션을 통해 피어 선택 최적화와 네트워크 효율성 향상

2. 적절한 청크 크기

- 기본 256KB 청크 크기가 대부분의 경우에 적합
- 네트워크 조건에 따라 청크 크기 조정 고려

3. 리소스 관리

- 사용하지 않는 세션은 적절히 정리
- 메모리 사용량 모니터링 및 제한 설정

4. 오류 처리

- 네트워크 오류와 타임아웃에 대한 적절한 재시도 로직
- 컨텍스트 취소를 통한 우아한 종료

문제 해결

일반적인 오류와 해결책

1. **블록을 찾을 수 없음**

- DHT 라우팅이 올바르게 설정되었는지 확인
- 제공자가 실제로 블록을 가지고 있는지 확인

2. **느린 다운로드 속도**

- 세션 사용 여부 확인
- 네트워크 연결 상태 점검
- 피어 수와 품질 확인

3. **메모리 사용량 증가**

- 블록스토어 크기 제한 설정
- 사용하지 않는 세션 정리
- 가비지 컬렉션 최적화

4. **연결 문제**

- 방화벽 및 NAT 설정 확인
- libp2p 연결 상태 모니터링
- 부트스트랩 노드 연결 상태 확인